Project 14 (Real-World Example) — InvoiceIQ-PDF-Invoice-Data-Extraction-And-Validation

Type: Document pipeline (load → extract → validate → report) · **Stack:** Python, LangChain, `langchain-community` loaders, OpenAI SDK, Pydantic · **Level:** Intermediate

Tagline: Read a messy invoice file into clean, typed data — then *check* it with code the model never runs, so the totals that don't add up are caught before anyone trusts them.

Charter (what this project is)

Every finance team drowns in invoices — PDFs and text files from dozens of vendors, each laid out differently, and someone has to key the numbers in and catch the ones where the total doesn't add up. **InvoiceIQ** reads an invoice file, extracts the fields that matter into a typed object, and validates them against deterministic rules before the data is trusted. The whole project is one idea: **the model reads; your code checks.**

Jump to: Project Structure · Setup · How testing works · Step 1 · Step 2 · Step 3

The Story — what you're building

A file comes in. A **loader** reads it into text (a PDF becomes a text blob; a `.txt` is read directly). An **extractor** sends that text to the model with `with_structured_output(Invoice)`, so the reply is already a typed `Invoice` — vendor, number, line items, total. A **validator** then runs plain Python: it sums `quantity * unit_price` across the line items and compares that to the stated total, and checks that required fields aren't blank. Each failed rule becomes a problem string. The **pipeline** returns the invoice plus any problems — empty means "cleared to post."

The lesson: an LLM is great at *reading* a messy document into a clean shape, and must never be *trusted* with the result. Extract with the model, verify with code — that split is what makes any document-AI system safe to act on.

Project Structure (the professional layout)

`main.py` at the repo root takes a file path; all code in `src/`; sample invoices in `samples/`.

```
<repo folder>/ (the InvoiceIQ repo — full name in the heading above)
├── samples/
```

```
│   ├── bad_total.txt           line items don't sum to the total
│   ├── clean.txt               a correct invoice
│   └── missing_number.txt      no invoice number
├── src/
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py              get_model()
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py
│   ├── __init__.py
│   ├── config.py
│   ├── exceptions.py
│   ├── loader.py               load_invoice_text()      (Step 3)
│   ├── pipeline.py             process_invoice(), Result (Step 3)
│   ├── schemas.py              Invoice, LineItem         (Step 1)
│   └── validator.py            validate_invoice()        (Step 1)
├── tests/
│   ├── integration/test_live_extract.py
│   └── unit/                   test_config, test_validator
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
├── README.md
└── requirements.txt
```

How the structure grows, step by step

- **Step 1** builds `config` / `exceptions` / `utils/logger`, `models/llm`, `src/schemas.py` (`Invoice`, `LineItem`) and `src/validator.py` — plus `test_config`, `test_validator`.
- **Step 2** adds `extract_invoice()` (`with_structured_output`) to `src/pipeline.py`.
- **Step 3** adds `src/loader.py`, finishes `process_invoice()`, writes `main.py` and the integration test.

Setup (do this once, before Step 1)

```
cd InvoiceIQ-PDF-Invoice-Data-Extraction-And-Validation
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install langchain-openai langchain-community openai pydantic python-dotenv
pip freeze > requirements.txt      # add pypdf if you load .pdf invoices
```

`.env.example` (copy to `.env`): `OPENAI_API_KEY=sk-...`, `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`. Every folder under `src/` gets an empty `__init__.py`. Write the three sample invoices in `samples/` (clean, bad total, missing number) so you have something to run against. `LICENSE` is the MIT text.

How testing works in this project

Two layers, one style (plain OK/WRONG scripts; this builds on Doc05/08, not pytest):

- **`tests/unit/`** — no model: `test_config`; `test_validator` (the deterministic checks, with hand-made `Invoice` objects — a clean one, a wrong total, a missing number).
- **`tests/integration/test_live_extract.py`** — the real model: a clean invoice clears; a wrong-total invoice is flagged.

Each step ends with a **Tests for this step** box.

Step 1 — The Typed Shape and the Deterministic Validator

What this step does: the `Invoice` / `LineItem` Pydantic models, and the validator — the heart of the project — built and tested with no model at all.

Why this step matters: the validator is the whole point. If you can prove it catches a wrong total and a missing field using hand-made objects, the rest (extraction) is just filling those objects in.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write config/exceptions/`utils/logger`, `models/llm` (the standard setup). 2. Write `src/schemas.py`: `LineItem(description, quantity, unit_price)` and `Invoice(vendor, invoice_number, line_items, total)` — **every field optional** (`Optional[...] = None`, `[]` for the list), so a missing field becomes a validation *problem*, never an invented value. 3. Write `src/validator.py`: `compute_subtotal(invoice)` (sum `quantity*unit_price` with a plain loop) and `validate_invoice(invoice) -> list[str]` — flag a missing number/vendor/total, no line items, and a total that doesn't match the subtotal (within a small tolerance).

Tests for this step: `test_validator` — a clean invoice has no problems; a wrong total is flagged; a missing number is flagged; `compute_subtotal` adds up. `test_config`.

Step 2 — Extract an Invoice From Text (the model's job)

What this step does: `extract_invoice(model, text)` — turn raw invoice text into a typed `Invoice` with `with_structured_output`.

Why this step matters: this is the only place the model is involved, and it's the easy part. Structured output guarantees the shape; your Step 1 validator decides whether to trust the values.

What's new vs. Step 1: `extract_invoice()`. **What stays the same:** the models and validator.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. In `src/pipeline.py`, write an `EXTRACT_PROMPT` that says: extract the fields, and **leave a field empty if it's not present — never invent a value**. 2. Write `extract_invoice(model, text)` = `prompt | model.with_structured_output(Invoice)`, then `.invoke({"text": text})`.

Tests for this step: proven in the integration test at the end (extraction needs a real model); for now, confirm `extract_invoice` returns an `Invoice` on a hand-written sample text.

Step 3 — The Loader and the Full Pipeline (final)

What this step does: the document loader (Doc08), the `process_invoice` pipeline that ties load → extract → validate together, `main.py`, and the integration test.

Why this step matters: this is where a real file turns into a checked result — and where PDFs stop being scary, because the loader hands you text no matter the format.

What's new vs. Step 2: `src/loader.py`, `process_invoice()`, `main.py`. **What stays the same:** the models, validator, extractor.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write `src/loader.py`: `load_invoice_text(path)` — read `.txt`/`.md` with `open()`, read `.pdf` with `PyPDFLoader(path).load()` (join the pages), raise a clear error for anything else. 2. Finish `src/pipeline.py`: `process_invoice(model, path)` loads the text, calls `extract_invoice`, runs `validate_invoice`, and returns a `Result(invoice, problems)`. 3. Write `main.py` (take a file path from `sys.argv`, print the fields and any problems) and `tests/integration/test_live_extract.py`.

Final Deliverable: a pipeline that reads an invoice file, extracts a typed `Invoice` with the model, and returns the data plus any problems a deterministic validator found.

Tests for this step: `test_live_extract` — `samples/clean.txt` clears; `samples/bad_total.txt` is flagged for a total mismatch.

Run, Test and Configure the Final Project

```
python main.py samples/clean.txt
python main.py samples/bad_total.txt
python -m tests.unit.test_config
python -m tests.unit.test_validator
python -m tests.integration.test_live_extract
```

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	Log verbosity

Checklist Before You Call This Done

- [ ] Every `Invoice` field is optional, so a missing one is a *problem*, not an invented value
- [ ] The validator catches a wrong total and a missing field — tested with no model
- [ ] Extraction uses `with_structured_output`, not hand-parsed JSON
- [ ] The loader reads `.txt` and `.pdf` into the same text shape
- [ ] You can explain why the model extracts but never gets the final say on a number

Publish It to GitHub

Name the repo `InvoiceIQ-PDF-Invoice-Data-Extraction-And-Validation`, add the `README.md`, commit `.env.example`, push. It's a document-AI pipeline any business understands — a strong portfolio piece.

Status

Not started.

Stuck? Ask for **Hint 1** or **Hint 2** before the solution. The validator (Step 1) is the real exercise — build it first.